IT408PC Java Programming Laboratory Manual

Instructor: Shafakhatullah Khan Mohammed

Contents

1. Experiment: IDE Familiarization + Debug Demo

IDE Familiarization + Debug Demo Program

Aim

To create and run a Java program and debug step-by-step using a small program containing **if–else-if** and a **for loop**.

Program

```
import java.util.Scanner;

public class DebugDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), sum = 0;

        for (int i = 1; i <= n; i++) {
            if (i % 2 == 0) sum += i;
            else sum += 1;
        }

        if (sum > 50) System.out.println("HIGH " + sum);
        else if (sum >= 20) System.out.println("MEDIUM " + sum);
        else System.out.println("LOW " + sum);

        sc.close();
    }
}
```

Output

Sample Input

```
10
```

Sample Output

```
MEDIUM 35
```

Sample Viva Questions

- What is the difference between Step Into and Step Over in debugging?
- Why is a breakpoint used?
- What happens if we do not close `Scanner`?
- What is the time complexity of the loop?
- What is refactoring in IDE and why is it useful?

2. Experiment: OOP Principles

2.1 Experiment: Encapsulation

Encapsulation Demonstration

Aim

To demonstrate **Encapsulation** using private fields and public methods (getters/setters).

Program

EncapsulationDemo.java

```
class BankAccount {
    private double balance; // encapsulated data

    public BankAccount(double initialBalance) {
        if (initialBalance < 0) initialBalance = 0;
        this.balance = initialBalance;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            return true;
        }
        return false;
    }
}

public class EncapsulationDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(1000);
        acc.deposit(500);
        boolean ok = acc.withdraw(2000);

        System.out.println("Withdraw success? " + ok);
        System.out.println("Balance: " + acc.getBalance());
    }
}
```

Output

```
Withdraw success? false
Balance: 1500.0
```

Sample Viva Questions

- What is encapsulation?

- Why do we keep fields `private`?
- What are getters and setters?
- What is data hiding?
- How does encapsulation improve maintainability?

2.2 Experiment: Inheritance

Inheritance Demonstration

Aim

To demonstrate **Inheritance** by extending a parent class into a child class.

Program

InheritanceDemo.java

```
class Vehicle {
    protected String brand;

    public Vehicle(String brand) {
        this.brand = brand;
    }

    public void start() {
        System.out.println(brand + " vehicle started");
    }
}

class Car extends Vehicle {
    private int seats;

    public Car(String brand, int seats) {
        super(brand);
        this.seats = seats;
    }

    public void showDetails() {
        System.out.println("Brand=" + brand + ", Seats=" + seats);
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Car c = new Car("Toyota", 5);
        c.start();
        c.showDetails();
    }
}
```

Output

```
Toyota vehicle started
Brand=Toyota, Seats=5
```

Sample Viva Questions

- What is inheritance?
- What is the use of `super()`?
- Difference between `protected` and `private`?
- Does Java support multiple inheritance with classes?
- What is an "is-a" relationship?

2.3 Experiment: Polymorphism (Overriding)

Polymorphism (Method Overriding) Demonstration

Aim

To demonstrate **Runtime Polymorphism** using method overriding.

Program

PolymorphismDemo.java

```
class Shape {
    public double area() {
        return 0;
    }
}

class Circle extends Shape {
    private double r;
    public Circle(double r) { this.r = r; }
    @Override public double area() { return 3.14159 * r * r; }
}

class Rectangle extends Shape {
    private double w, h;
    public Rectangle(double w, double h) { this.w = w; this.h = h; }
    @Override public double area() { return w * h; }
}

public class PolymorphismDemo {
    public static void main(String[] args) {
        Shape s1 = new Circle(2);
        Shape s2 = new Rectangle(3, 4);

        System.out.println("Circle area: " + s1.area());
        System.out.println("Rectangle area: " + s2.area());
    }
}
```

Output

```
Circle area: 12.56636
Rectangle area: 12.0
```

Sample Viva Questions

- What is polymorphism?
- Difference between method overloading and overriding?
- What is dynamic method dispatch?
- Why use parent reference for child objects?
- Can static methods be overridden?

2.4 Experiment: Abstraction

Abstraction Demonstration

Aim

To demonstrate **Abstraction** using an abstract class and abstract method.

Program

AbstractionDemo.java

```
abstract class Payment {
    public abstract void pay(double amount);

    public void receipt(double amount) {
        System.out.println("Receipt generated for amount: " + amount);
    }
}

class CardPayment extends Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using Card");
    }
}

class UPIPayment extends Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using UPI");
    }
}

public class AbstractionDemo {
    public static void main(String[] args) {
        Payment p1 = new CardPayment();
        p1.pay(500);
        p1.receipt(500);

        Payment p2 = new UPIPayment();
        p2.pay(250);
        p2.receipt(250);
    }
}
```

Output

```
Paid 500.0 using Card
Receipt generated for amount: 500.0
Paid 250.0 using UPI
Receipt generated for amount: 250.0
```

Sample Viva Questions

- What is abstraction?
- Difference between abstract class and interface?
- Can an abstract class have constructors?
- Can an abstract class contain non-abstract methods?
- Why is abstraction important in large systems?

3. Experiment: Exception Handling

3.1 Experiment: Checked Exception

Checked Exception Handling Demo

Aim

To handle a **checked exception** (`IOException`) using try-catch.

Program

CheckedExceptionDemo.java

```
import java.io.*;

public class CheckedExceptionDemo {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new FileReader("missing.txt"))
            ) {
            System.out.println(br.readLine());
        } catch (IOException e) {
            System.out.println("Checked exception handled: " + e.getClass().
                getSimpleName());
        }
    }
}
```

Output

```
Checked exception handled: FileNotFoundException
```

Sample Viva Questions

- What is a checked exception?
- Why does Java force handling checked exceptions?
- What is try-with-resources?
- Difference between `throw` and `throws`?
- What is the use of `finally`?

3.2 Experiment: Unchecked Exception

Unchecked Exception Handling Demo

Aim

To handle an **unchecked exception** (`ArithmeticException`).

Program

UncheckedExceptionDemo.java

```
public class UncheckedExceptionDemo {
    public static void main(String[] args) {
        try {
```

```
            int x = 10 / 0;
            System.out.println(x);
        } catch (ArithmeticException e) {
            System.out.println("Unchecked exception handled: " + e);
        }
    }
}
```

Output

```
Unchecked exception handled: java.lang.ArithmeticException: / by zero
```

Sample Viva Questions

- What is an unchecked exception?
- Why are unchecked exceptions not forced by compiler?
- Is `NullPointerException` checked or unchecked?
- What is exception propagation?
- Can we catch multiple exceptions in a single catch?

3.3 Experiment: Custom Exception

Custom Exception in Real Scenario (Insufficient Balance)

Aim

To create and use a **custom exception** in a realistic scenario.

Program

CustomExceptionDemo.java

```
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String msg) { super(msg); }
}

class Wallet {
    private double balance;
    public Wallet(double balance) { this.balance = balance; }

    public void pay(double amount) throws InsufficientBalanceException {
        if (amount > balance)
            throw new InsufficientBalanceException("Balance=" + balance + ",
                required=" + amount);
        balance -= amount;
    }

    public double getBalance() { return balance; }
}

public class CustomExceptionDemo {
    public static void main(String[] args) {
        Wallet w = new Wallet(500);

```

```
        try {
            w.pay(700);
        } catch (InsufficientBalanceException e) {
            System.out.println("Custom exception handled: " + e.getMessage());
        }

        System.out.println("Final balance: " + w.getBalance());
    }
}
```

Output

```
Custom exception handled: Balance=500.0, required=700.0
Final balance: 500.0
```

Sample Viva Questions

- Why do we create custom exceptions?
- Difference between `Exception` and `RuntimeException`?
- When should a custom exception be checked?
- What is the use of `super(message)`?
- Can we create custom unchecked exceptions?

4. Experiment: RandomAccessFile

4.1 Experiment: Write Records

RandomAccessFile: Write Records

Aim

To write fixed-length records using `RandomAccessFile`.

Program

RAFWriteDemo.java

```
import java.io.*;

public class RAFWriteDemo {
    static final int NAME_CHARS = 20;

    static void writeFixedString(RandomAccessFile raf, String s) throws
        IOException {
        StringBuilder sb = new StringBuilder(s);
        sb.setLength(NAME_CHARS);
        raf.writeChars(sb.toString());
    }

    public static void main(String[] args) throws Exception {
        try (RandomAccessFile raf = new RandomAccessFile("emp.dat", "rw")) {
            raf.setLength(0);

            raf.writeInt(101); writeFixedString(raf, "Asha");
            raf.writeInt(102); writeFixedString(raf, "Ravi");

            System.out.println("Records written successfully.");
        }
    }
}
```

Output

```
Records written successfully.
```

Sample Viva Questions

- What is `RandomAccessFile` used for?
- Why use fixed-length strings?
- What does `raf.setLength(0)` do?
- Difference between sequential and random access?
- What does mode `"rw"` mean?

4.2 Experiment: Read by Index

RandomAccessFile: Read Record at Index

Aim

To read a record from a specific index using `seek()`.

Program

RAFReadDemo.java

```
import java.io.*;

public class RAFReadDemo {
    static final int NAME_CHARS = 20;
    static final int RECORD_SIZE = 4 + 2 * NAME_CHARS;

    static String readFixedString(RandomAccessFile raf) throws IOException {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < NAME_CHARS; i++) sb.append(raf.readChar());
        return sb.toString().trim();
    }

    static void writeFixedString(RandomAccessFile raf, String s) throws
        IOException {
        StringBuilder sb = new StringBuilder(s);
        sb.setLength(NAME_CHARS);
        raf.writeChars(sb.toString());
    }

    public static void main(String[] args) throws Exception {
        try (RandomAccessFile raf = new RandomAccessFile("emp.dat", "rw")) {
            raf.setLength(0);
            raf.writeInt(101); writeFixedString(raf, "Asha");
            raf.writeInt(102); writeFixedString(raf, "Ravi");
            raf.writeInt(103); writeFixedString(raf, "Meera");

            int index = 1; // read 2nd record
            raf.seek((long) index * RECORD_SIZE);

            int id = raf.readInt();
            String name = readFixedString(raf);

            System.out.println("Record[" + index + "]: " + id + " " + name);
        }
    }
}
```

Output

```
Record[1]: 102 Ravi
```

Sample Viva Questions

- What does `seek()` do?
- How is `RECORD_SIZE` calculated?

- What happens if we seek beyond file length?
- Why fixed-size records help random access?
- Can we update a record without rewriting the entire file?

4.3 Experiment: Update In-place

RandomAccessFile: Update Record (In-place)

Aim

To update a record directly at its location using `seek()`.

Program

RAFUpdateDemo.java

```
import java.io.*;

public class RAFUpdateDemo {
    static final int NAME_CHARS = 20;
    static final int RECORD_SIZE = 4 + 2 * NAME_CHARS;

    static void writeFixedString(RandomAccessFile raf, String s) throws
        IOException {
        StringBuilder sb = new StringBuilder(s);
        sb.setLength(NAME_CHARS);
        raf.writeChars(sb.toString());
    }

    static String readFixedString(RandomAccessFile raf) throws IOException {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < NAME_CHARS; i++) sb.append(raf.readChar());
        return sb.toString().trim();
    }

    public static void main(String[] args) throws Exception {
        try (RandomAccessFile raf = new RandomAccessFile("emp.dat", "rw")) {
            raf.setLength(0);
            raf.writeInt(101); writeFixedString(raf, "Alice");
            raf.writeInt(102); writeFixedString(raf, "John");

            int index = 1; // update 2nd record
            raf.seek((long) index * RECORD_SIZE + 4); // skip int id
            writeFixedString(raf, "John Hayns");

            raf.seek((long) index * RECORD_SIZE);
            int id = raf.readInt();
            String name = readFixedString(raf);

            System.out.println("Updated Record[" + index + "]: " + id + " " + name)
                ;
        }
    }
}
```

Output

```
Updated Record[1]: 102 John Hayns
```

Sample Viva Questions

- Why do we add `+4` when updating name?
- What is an in-place update?
- What if new string is longer than fixed size?
- Can `RandomAccessFile` handle binary data?
- When is RAF preferred over normal file streams?

5. Experiment: Collections Framework

Note: For these sub-experiments, use package `collections.lab` and compile/run each file separately.

5.1 Experiment: List Demo (ArrayList, LinkedList, Stack)

Collections: List Demo (ArrayList, LinkedList, Stack)

Aim

To demonstrate List operations and stack behavior.

Program

collections/lab/ListDemo.java

```
package collections.lab;

import java.util.*;

public class ListDemo {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(3);
        list.add(1);
        list.add(2);
        list.add(1, 99);
        list.remove(Integer.valueOf(1));
        Collections.sort(list);
        System.out.println("ArrayList: " + list);

        LinkedList<String> ll = new LinkedList<>();
        ll.addFirst("VIP");
        ll.addLast("NORMAL");
        System.out.println("LinkedList peekFirst: " + ll.peekFirst());
        System.out.println("LinkedList pollFirst: " + ll.pollFirst());
        System.out.println("LinkedList: " + ll);

        Stack<String> st = new Stack<>();
        st.push("A"); st.push("B");
        System.out.println("Stack peek: " + st.peek());
        System.out.println("Stack pop: " + st.pop());
    }
}
```

Output

```
ArrayList: [2, 3, 99]
LinkedList peekFirst: VIP
LinkedList pollFirst: VIP
LinkedList: [NORMAL]
Stack peek: B
Stack pop: B
```

Viva

- Difference between ArrayList and LinkedList?
- Why is Stack considered legacy?
- Difference between `remove(index)` and `remove(Object)`?
- What does `Collections.sort()` do?
- Time complexity of `get()` in ArrayList?

5.2 Experiment: Queue/Deque Demo (ArrayDeque, PriorityQueue)

Collections: Queue/Deque Demo (ArrayDeque, PriorityQueue)

Aim

To demonstrate queue operations using `ArrayDeque` and heap behavior using `PriorityQueue`.

Program

collections/lab/QueueDemo.java

```
package collections.lab;

import java.util.*;

public class QueueDemo {
    public static void main(String[] args) {
        ArrayDeque<Integer> dq = new ArrayDeque<>();
        dq.offerLast(10);
        dq.offerFirst(5);
        dq.offerLast(20);
        System.out.println("ArrayDeque: " + dq);
        System.out.println("peekFirst: " + dq.peekFirst());
        System.out.println("pollLast: " + dq.pollLast());
        System.out.println("After pollLast: " + dq);

        PriorityQueue<Integer> pq = new PriorityQueue<>(); // min-heap
        pq.offer(3); pq.offer(1); pq.offer(2);
        System.out.println("PriorityQueue peek: " + pq.peek());
        System.out.println("PriorityQueue poll: " + pq.poll());
        System.out.println("PriorityQueue after: " + pq);
    }
}
```

Output

```
ArrayDeque: [5, 10, 20]
peekFirst: 5
pollLast: 20
After pollLast: [5, 10]
PriorityQueue peek: 1
PriorityQueue poll: 1
PriorityQueue after: [2, 3]
```

Viva

- Why is ArrayDeque often faster than LinkedList as a deque?
- What ordering does PriorityQueue maintain?
- Is PriorityQueue fully sorted when printed?
- Difference between `peek()` and `poll()`?
- What is a heap?

5.3 Experiment: Set Demo (HashSet, LinkedHashSet, TreeSet)

Collections: Set Demo (HashSet, LinkedHashSet, TreeSet)

Aim

To demonstrate uniqueness, insertion-order, and sorted-order set behavior.

Program

collections/lab/SetDemo.java

```
package collections.lab;

import java.util.*;

public class SetDemo {
    public static void main(String[] args) {
        HashSet<Integer> hs = new HashSet<>();
        hs.add(3); hs.add(3); hs.add(1);
        System.out.println("HashSet: " + hs);

        LinkedHashSet<Integer> lhs = new LinkedHashSet<>();
        lhs.add(3); lhs.add(3); lhs.add(1); lhs.add(2);
        System.out.println("LinkedHashSet: " + lhs);

        TreeSet<Integer> ts = new TreeSet<>();
        ts.add(5); ts.add(1); ts.add(3);
        System.out.println("TreeSet: " + ts);
        System.out.println("first/last: " + ts.first() + "/" + ts.last());
        System.out.println("ceiling(4): " + ts.ceiling(4));
    }
}
```

Output

```
HashSet: [1, 3]
LinkedHashSet: [3, 1, 2]
TreeSet: [1, 3, 5]
first/last: 1/5
ceiling(4): 5
```

Sample Viva Questions

- Difference between HashSet and TreeSet?
- What order does LinkedHashSet preserve?

- Average complexity of add/contains in HashSet?
- What does `ceiling()` do?
- What happens when duplicates are inserted into a Set?

5.4 Experiment: Map Demo (HashMap, LinkedHashMap, TreeMap)

Collections: Map Demo (HashMap, LinkedHashMap, TreeMap)

Aim

To demonstrate key-value storage and ordering behavior in maps.

Program

collections/lab/MapDemo.java

```
package collections.lab;

import java.util.*;

public class MapDemo {
    public static void main(String[] args) {
        HashMap<String, Integer> hm = new HashMap<>();
        hm.put("A", 2);
        hm.put("B", hm.getOrDefault("B", 0) + 1);
        System.out.println("HashMap: " + hm);

        LinkedHashMap<String, Integer> lhm = new LinkedHashMap<>();
        lhm.put("first", 1);
        lhm.put("second", 2);
        System.out.println("LinkedHashMap: " + lhm);

        TreeMap<Integer, String> tm = new TreeMap<>();
        tm.put(10, "ten");
        tm.put(5, "five");
        System.out.println("TreeMap: " + tm);
        System.out.println("floorKey(7): " + tm.floorKey(7));
    }
}
```

Output

```
HashMap: {A=2, B=1}
LinkedHashMap: {first=1, second=2}
TreeMap: {5=five, 10=ten}
floorKey(7): 5
```

Sample Viva Questions

- Difference between HashMap and LinkedHashMap?
- What ordering does TreeMap maintain?
- What is `getOrDefault()` used for?
- Can HashMap store null keys/values?

- What is `floorKey()`?

5.5 Experiment: Comparable vs Comparator

Comparable vs Comparator Demo

Aim

To demonstrate natural ordering (`Comparable`) and custom ordering (`Comparator`).

Program

collections/lab/SortDemo.java

```
package collections.lab;

import java.util.*;

class Employee implements Comparable<Employee> {
    int id;
    String name;
    int score;

    Employee(int id, String name, int score) {
        this.id = id; this.name = name; this.score = score;
    }

    @Override
    public int compareTo(Employee o) { // natural order: id ascending
        return Integer.compare(this.id, o.id);
    }

    @Override
    public String toString() {
        return "{id=" + id + ", name=" + name + ", score=" + score + "}";
    }
}

public class SortDemo {
    public static void main(String[] args) {
        List<Employee> emps = Arrays.asList(
            new Employee(2, "Ravi", 80),
            new Employee(1, "Asha", 90),
            new Employee(3, "Meera", 90)
        );

        // Comparator: score desc then name asc
        emps.sort(Comparator
                .comparingInt((Employee e) -> e.score).reversed()
                .thenComparing(e -> e.name));

        System.out.println("Comparator sort: " + emps);

        // Comparable: TreeSet uses natural order by id
        TreeSet<Employee> set = new TreeSet<>(emps);
        System.out.println("Comparable (TreeSet): " + set);
    }
}
```

Output

```
Comparator sort: [{id=1, name=Asha, score=90}, {id=3, name=Meera, score=90}, {id=2, name=Ravi, score=80}]
Comparable (TreeSet): [{id=1, name=Asha, score=90}, {id=2, name=Ravi, score=80}, {id=3, name=Meera, score=90}]
```

Sample Viva Questions

- Difference between Comparable and Comparator?
- Where is `compareTo()` defined?
- Can a class have multiple Comparator implementations?
- What happens if compareTo returns 0 for different objects in TreeSet?
- Why are tie-breakers important in sorting?

6. Experiment: Thread Synchronization

6.1 Experiment: Without Synchronization

Ticket Booking WITHOUT Synchronization

Aim

To show race condition possibility when multiple threads access shared data.

Program

TicketNoSyncDemo.java

```
class CounterNoSync {
    int available = 5;

    boolean book(String user, int seats) {
        System.out.println(user + " checking... Available=" + available);
        if (seats <= available) {
            try { Thread.sleep(50); } catch (InterruptedException ignored) {}
            available -= seats;
            System.out.println("Booked for " + user + ". Remaining=" + available);
            return true;
        }
        System.out.println("Failed for " + user);
        return false;
    }
}

public class TicketNoSyncDemo {
    public static void main(String[] args) throws Exception {
        CounterNoSync c = new CounterNoSync();
        Thread t1 = new Thread(() -> c.book("UserA", 3));
        Thread t2 = new Thread(() -> c.book("UserB", 3));

        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("Final Remaining=" + c.available);
    }
}
```

Output

Output can vary between runs (race condition may cause inconsistent results).

Sample Viva Questions

- What is a race condition?
- Why is output not consistent every run?
- What is a critical section?
- What is the shared resource here?
- How can we prevent race conditions?

6.2 Experiment: With Synchronization

Ticket Booking WITH Synchronization

Aim

To prevent race condition using the `synchronized` keyword.

Program

TicketSyncDemo.java

```
class CounterSync {
    private int available = 5;

    public synchronized boolean book(String user, int seats) {
        System.out.println(user + " checking... Available=" + available);
        if (seats <= available) {
            try { Thread.sleep(50); } catch (InterruptedException ignored) {}
            available -= seats;
            System.out.println("Booked for " + user + ". Remaining=" + available);
            return true;
        }
        System.out.println("Failed for " + user);
        return false;
    }
}

public class TicketSyncDemo {
    public static void main(String[] args) throws Exception {
        CounterSync c = new CounterSync();
        Thread t1 = new Thread(() -> c.book("UserA", 3));
        Thread t2 = new Thread(() -> c.book("UserB", 3));

        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("Done");
    }
}
```

Output

Consistent safe booking. One thread may fail if seats are insufficient.

Sample Viva Questions

- What does `synchronized` do internally?
- What is an object monitor/lock?
- Difference between synchronized method and synchronized block?
- Can deadlock happen? How?
- What is the use of `join()`?

7. Experiment: JDBC CRUD on Student Table

Database Setup (Run Once)

student_setup.sql

```
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE IF NOT EXISTS student (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  dept VARCHAR(30) NOT NULL
);
```

Note: Add MySQL JDBC driver (Connector/J) to your project classpath. Update URL/USER/-PASS in programs.

7.1 Experiment: INSERT

JDBC: CREATE/INSERT Student

Aim

To insert a row into `student` using `PreparedStatement`.

Program

StudentInsertDemo.java

```
import java.sql.*;

public class StudentInsertDemo {
    static final String URL = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root";
    static final String PASS = "your_password";

    public static void main(String[] args) throws Exception {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {
            String sql = "INSERT INTO student(id,name,dept) VALUES(?,?,?)";
            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setInt(1, 1);
                ps.setString(2, "Asha");
                ps.setString(3, "CSE");
                System.out.println("Inserted rows: " + ps.executeUpdate());
            }
        }
    }
}
```

Output

```
Inserted rows: 1
```

Sample Viva Questions

- Why use `PreparedStatement`?
- What is JDBC?
- What is SQL injection?
- What is a JDBC URL?
- What does `executeUpdate()` return?

7.2 Experiment: SELECT

JDBC: READ/SELECT Students

Aim

To retrieve rows from the table using `SELECT`.

Program

StudentSelectDemo.java

```
import java.sql.*;

public class StudentSelectDemo {
    static final String URL = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root";
    static final String PASS = "your_password";

    public static void main(String[] args) throws Exception {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
             Statement st = con.createStatement();
             ResultSet rs = st.executeQuery("SELECT id,name,dept FROM student ORDER
                 BY id")) {

            while (rs.next()) {
                System.out.println(rs.getInt("id") + " " +
                                   rs.getString("name") + " " +
                                   rs.getString("dept"));
            }
        }
    }
}
```

Output

```
1 Asha CSE
```

Sample Viva Questions

- What is `ResultSet`?
- Difference between `Statement` and `PreparedStatement`?
- What does `executeQuery()` return?
- What is a cursor in ResultSet?

- Why close DB resources?

7.3 Experiment: UPDATE

JDBC: UPDATE Student

Aim

To update an existing record using `UPDATE`.

Program

StudentUpdateDemo.java

```
import java.sql.*;

public class StudentUpdateDemo {
    static final String URL = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root";
    static final String PASS = "your_password";

    public static void main(String[] args) throws Exception {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {
            String sql = "UPDATE student SET dept=? WHERE id=?";
            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setString(1, "IT");
                ps.setInt(2, 1);
                System.out.println("Updated rows: " + ps.executeUpdate());
            }
        }
    }
}
```

Output

```
Updated rows: 1
```

Sample Viva Questions

- Why is WHERE clause important in UPDATE?
- What happens if no rows match the WHERE clause?
- What is auto-commit?
- What is a transaction?
- Why parameter binding is safer?

7.4 Experiment: DELETE

JDBC: DELETE Student

Aim

To delete a record using `DELETE`.

Program

StudentDeleteDemo.java

```
import java.sql.*;

public class StudentDeleteDemo {
    static final String URL = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root";
    static final String PASS = "your_password";

    public static void main(String[] args) throws Exception {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {
            String sql = "DELETE FROM student WHERE id=?";
            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setInt(1, 1);
                System.out.println("Deleted rows: " + ps.executeUpdate());
            }
        }
    }
```

Output

```
Deleted rows: 1
```

Sample Viva Questions

- What is the risk of DELETE without WHERE?
- Difference between DELETE and TRUNCATE?
- What is rollback?
- What is referential integrity?
- How to safely delete data?

8. Experiment: Swing Calculator

8.1 Experiment: Layout Only

Calculator UI using GridLayout (Layout Only)

Aim

To design calculator GUI using Swing and GridLayout.

Program

CalculatorLayoutOnly.java

```
import javax.swing.*;
import java.awt.*;

public class CalculatorLayoutOnly extends JFrame {
    public CalculatorLayoutOnly() {
        super("Calculator Layout");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(320, 420);
        setLayout(new BorderLayout(8, 8));

        JTextField display = new JTextField();
        display.setEditable(false);
        display.setHorizontalAlignment(SwingConstants.RIGHT);
        display.setFont(new Font("Arial", Font.BOLD, 22));
        add(display, BorderLayout.NORTH);

        JPanel grid = new JPanel(new GridLayout(4, 4, 6, 6));
        String[] keys = {"7","8","9","/","4","5","6","*","1","2","3","-","0","C
            ","=","+"};

        for (String k : keys) {
            JButton b = new JButton(k);
            b.setFont(new Font("Arial", Font.BOLD, 18));
            grid.add(b);
        }

        add(grid, BorderLayout.CENTER);
        setLocationRelativeTo(null);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new CalculatorLayoutOnly().setVisible(
            true));
    }
}
```

Output

A calculator UI opens with buttons; no calculations implemented.

Sample Viva Questions

- What is GridLayout?

- Why use BorderLayout + GridLayout together?
- What is EDT in Swing?
- Difference between JFrame and JPanel?
- What is ActionListener used for?

8.2 Experiment: Basic + and -

Calculator for + and - (Basic Operations)

Aim

To implement basic operations (+ and -) using button events.

Program

CalculatorAddSub.java

```
import javax.swing.*;
import java.awt.*;

public class CalculatorAddSub extends JFrame {
    private final JTextField display = new JTextField();
    private double first = 0;
    private String op = null;
    private boolean clear = false;

    public CalculatorAddSub() {
        super("Calculator + -");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(320, 420);
        setLayout(new BorderLayout(8, 8));

        display.setEditable(false);
        display.setHorizontalAlignment(SwingConstants.RIGHT);
        display.setFont(new Font("Arial", Font.BOLD, 22));
        add(display, BorderLayout.NORTH);

        JPanel grid = new JPanel(new GridLayout(4, 4, 6, 6));
        String[] keys = {"7","8","9","+","4","5","6","-","1","2","3","=","0","C
            ","",""};
        for (String k : keys) {
            JButton b = new JButton(k);
            b.setFont(new Font("Arial", Font.BOLD, 18));
            b.addActionListener(e -> onPress(k));
            grid.add(b);
        }

        add(grid, BorderLayout.CENTER);
        setLocationRelativeTo(null);
    }

    private void onPress(String k) {
        if (k.isEmpty()) return;

        if (k.equals("C")) {
            display.setText("");
            first = 0; op = null; clear = false;
```

```
            return;
        }

        if (k.equals("+") || k.equals("-")) {
            if (!display.getText().isEmpty()) {
                first = Double.parseDouble(display.getText());
                op = k;
                clear = true;
            }
            return;
        }

        if (k.equals("=")) {
            if (op == null || display.getText().isEmpty()) return;
            double second = Double.parseDouble(display.getText());
            double ans = op.equals("+") ? first + second : first - second;
            display.setText(String.valueOf(ans));
            op = null;
            clear = true;
            return;
        }

        // digit
        if (clear) { display.setText(""); clear = false; }
        display.setText(display.getText() + k);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new CalculatorAddSub().setVisible(true));
    }
}
```

Output

GUI calculator performs addition and subtraction.

Sample Viva Questions

- What is event-driven programming?
- Why is a clear flag used?
- How are button clicks handled in Swing?
- What is parsing (`Double.parseDouble`)?
- How to restrict invalid inputs in calculator?

8.3 Experiment: Divide-by-Zero Handling

Calculator with / and Divide-by-Zero Handling

Aim

To handle exceptions like divide-by-zero during calculation.

Program

CalculatorDivideByZero.java

```
import javax.swing.*;
import java.awt.*;

public class CalculatorDivideByZero extends JFrame {
    private final JTextField display = new JTextField();
    private double first = 0;
    private String op = null;
    private boolean clear = false;

    public CalculatorDivideByZero() {
        super("Calculator / Exception");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(320, 420);
        setLayout(new BorderLayout(8, 8));
        display.setEditable(false);
        display.setHorizontalAlignment(SwingConstants.RIGHT);
        display.setFont(new Font("Arial", Font.BOLD, 22));
        add(display, BorderLayout.NORTH);
        JPanel grid = new JPanel(new GridLayout(4, 4, 6, 6));
        String[] keys = {"7","8","9","/","4","5","6","*","1","2","3","-","0","C
            ","=","+"};

        for (String k : keys) {
            JButton b = new JButton(k);
            b.setFont(new Font("Arial", Font.BOLD, 18));
            b.addActionListener(e -> onPress(k));
            grid.add(b);
        }
        add(grid, BorderLayout.CENTER);
        setLocationRelativeTo(null);
    }

    private void onPress(String k) {
        try {
            if (k.equals("C")) {
                display.setText("");
                first = 0; op = null; clear = false;
                return;
            }

            if ("+-*/".contains(k)) {
                if (!display.getText().isEmpty()) {
                    first = Double.parseDouble(display.getText());
                    op = k;
                    clear = true;
                }
                return;
```

```
            }

            if (k.equals("=")) {
                if (op == null || display.getText().isEmpty()) return;
                double second = Double.parseDouble(display.getText());

                double ans;
                if (op.equals("+")) ans = first + second;
                else if (op.equals("-")) ans = first - second;
                else if (op.equals("*")) ans = first * second;
                else { // "/"
                    if (second == 0) throw new ArithmeticException("Divide by zero
                        ");
                    ans = first / second;
                }

                display.setText(String.valueOf(ans));
                op = null;
                clear = true;
                return;
            }

            // digit
            if (clear) { display.setText(""); clear = false; }
            display.setText(display.getText() + k);

        } catch (NumberFormatException ex) {
            display.setText("Error");
            op = null;
        } catch (ArithmeticException ex) {
            display.setText(ex.getMessage());
            op = null;
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new CalculatorDivideByZero().setVisible(
            true));
    }
}
```

Output

When dividing by zero, display shows:

```
Divide by zero
```

Viva

- Why catch `ArithmeticException` separately?
- What is `NumberFormatException`?
- How do we prevent application crash in event handler?
- What is input validation?
- Why GUI error handling is important?

9. Experiment: Mouse Events

9.1 Experiment: MouseAdapter

MouseListener using MouseAdapter

Aim

To handle mouse click/press/release/enter/exit events using `MouseAdapter`.

Program

MouseAdapterDemo.java

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MouseAdapterDemo extends JFrame {
    private final JLabel label = new JLabel("Do mouse actions...", SwingConstants.
        CENTER);

    public MouseAdapterDemo() {
        super("MouseAdapter Demo");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(500, 300);
        label.setFont(new Font("Arial", Font.BOLD, 20));
        add(label, BorderLayout.CENTER);

        addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) { label.setText("mouseClicked");
                 }
            public void mousePressed(MouseEvent e) { label.setText("mousePressed");
                 }
            public void mouseReleased(MouseEvent e) { label.setText("mouseReleased
                "); }
            public void mouseEntered(MouseEvent e) { label.setText("mouseEntered");
                 }
            public void mouseExited(MouseEvent e) { label.setText("mouseExited"); }
        });

        setLocationRelativeTo(null);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new MouseAdapterDemo().setVisible(true));
    }
}
```

Output

Label updates with the event name when the event occurs.

Sample Viva Questions

- What is an adapter class?
- Why not implement MouseListener directly?

- Which event triggers when mouse enters the window?
- What is `MouseEvent`?
- What is the Swing EDT?

9.2 Experiment: MouseMotionAdapter

MouseMotionListener using MouseMotionAdapter

Aim

To handle mouse move and drag events using `MouseMotionAdapter`.

Program

MouseMotionAdapterDemo.java

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MouseMotionAdapterDemo extends JFrame {
    private final JLabel label = new JLabel("Move/Drag mouse...", SwingConstants.
        CENTER);

    public MouseMotionAdapterDemo() {
        super("MouseMotionAdapter Demo");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(500, 300);
        label.setFont(new Font("Arial", Font.BOLD, 20));
        add(label, BorderLayout.CENTER);

        addMouseMotionListener(new MouseMotionAdapter() {
            public void mouseMoved(MouseEvent e) { label.setText("mouseMoved"); }
            public void mouseDragged(MouseEvent e) { label.setText("mouseDragged");
                 }
        });

        setLocationRelativeTo(null);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new MouseMotionAdapterDemo().setVisible(
            true));
    }
}
```

Output

Label changes continuously while moving or dragging the mouse.

Sample Viva Questions

- Difference between MouseListener and MouseMotionListener?
- Which event fires continuously during movement?
- What is dragging?

- What is an event listener?
- Why use adapter classes?

9.3 Experiment: All Mouse Events Combined

All Mouse Events: Show Event Name at Center

Aim

To handle all mouse events and display the event name at the center of the window.

Program

MouseEventsAllDemo.java

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MouseEventsAllDemo extends JFrame {
    private final JLabel label = new JLabel("Perform mouse actions...",
        SwingConstants.CENTER);

    public MouseEventsAllDemo() {
        super("Mouse Events All Demo");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(500, 300);

        label.setFont(new Font("Arial", Font.BOLD, 20));
        add(label, BorderLayout.CENTER);

        addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) { label.setText("mouseClicked at
                " + e.getX() + "," + e.getY()); }
            public void mousePressed(MouseEvent e) { label.setText("mousePressed");
                }
            public void mouseReleased(MouseEvent e) { label.setText("mouseReleased
                "); }
            public void mouseEntered(MouseEvent e) { label.setText("mouseEntered");
                }
            public void mouseExited(MouseEvent e) { label.setText("mouseExited"); }
        });

        addMouseMotionListener(new MouseMotionAdapter() {
            public void mouseMoved(MouseEvent e) { label.setText("mouseMoved"); }
            public void mouseDragged(MouseEvent e) { label.setText("mouseDragged");
                }
        });

        setLocationRelativeTo(null);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new MouseEventsAllDemo().setVisible(true)
            );
    }
}
```

Output

A window opens; label updates to event name on mouse actions.

Sample Viva Questions

- What is the benefit of adapter classes?
- Difference between click and press?
- Why does mouseMoved fire frequently?
- Why place label in BorderLayout.CENTER?
- What is `invokeLater()` used for?